Purpose

INT-0 — intake/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/intake/INT-0_purpose.md`

---
library: intake
id: INT-0
schema_version: 1.0.0
status: draft

INT-0 — intake/ Purpose + Boundaries

Purpose
`intake/` defines the **Intake Wizard contract**: the schemas, enums, validation rules, and determinism rules that govern how Axion collects inputs and turns them into a stable submission record.

This library is the single source of truth for:
- the intake form spec (pages/sections/fields)
- allowed values (enums, controlled vocabularies)
- validation rules (field-level + cross-field)
- submission record format (raw + normalized)
- determinism rules (normalization, stable ordering, id generation)

What it governs (in scope)
- **Form spec**: page order, field definitions, dependencies/visibility rules
- **Field types**: text/enum/multi-select/file refs/etc.
- **Field enums**: canonical allowed values and aliases
- **Validation rules**:
  - required fields
  - cross-field constraints
  - conditional requirements
- **Submission artifacts**:
  - submission record (raw answers)
  - normalized input (canonical form for downstream stages)
- **Determinism rules**:
  - normalization rules (case, whitespace, canonical slugs)
  - stable sorting of lists
  - stable IDs for submissions and attachments

What it does NOT govern (out of scope)
- Canonical spec structure/relationships and unknowns handling → `canonical/`
- Standards packs and standards resolution → `standards/`
- Templates registry/selection/render rules → `templates/`
- Knowledge selection rules → `knowledge/`
- Gate DSL semantics → `gates/`
- Pipeline stage order and run manifest → `orchestration/`
- Workspace/pins/adapters/quotas/policy hook plumbing → `system/`
- Risk class meaning and override rules → `policy/`

Consumers
- Wizard UI (to render the form deterministically)
- Ingest/normalize stage (S1) for normalization
- Intake validation stage (S2) for enforcement + reporting
- Orchestrator (to block/pause on invalid intake)

Determinism requirements
- Same user answers → same normalized output (stable ordering, stable slugs).
- Form spec version is pinned and recorded in run manifest.
- Enum resolution is deterministic (alias → canonical).
- Validation produces stable error codes and pointers.

Piece B — Boundary checklist

File: `Axion/libraries/intake/INT-0_boundary_checklist.md`

---
library: intake
id: INT-0a
schema_version: 1.0.0
status: draft

INT-0a — Boundary Checklist

Belongs in `intake/` if it answers:
- “What questions do we ask and how are they structured?”
- “What values are allowed for this field?”
- “How do we validate this submission?”
- “How do we normalize answers deterministically into canonical input?”

Does NOT belong in `intake/` if it answers:
- “How do we build the canonical spec?” (canonical)
- “Which standards apply?” (standards)
- “Which templates are selected?” (templates)
- “How do gates evaluate predicates?” (gates)
- “What is the pipeline stage order?” (orchestration)

Say **“next”** for **INT-1 Form spec model (pages, fields, dependencies)**.

Form spec model

INT-1 — Form spec model (pages, fields, dependencies)

Piece A — Form spec model (concept)

File: `Axion/libraries/intake/INT-1_form_spec_model.md`

---
library: intake
id: INT-1
schema_version: 1.0.0
status: draft

INT-1 — Form Spec Model

Purpose
Define a versioned intake wizard spec that is:
- deterministic (stable page/field ordering)
- machine-readable (UI can render it)
- validatable (submission can be checked against it)

Core entities
- **Form spec**: top-level container (versioned)
- **Page**: ordered page of the wizard
- **Field**: question/input on a page
- **Visibility rule**: when a page/field is shown
- **Dependency**: field requirements conditioned on other answers

Field types (minimum)
- text
- textarea
- number
- boolean
- enum (single select)
- multi_enum (multi select)
- date
- file_ref (uploaded file pointer)
- json (advanced)

Required properties
Form spec:
- form_id

- version
- pages[] (ordered)
- created_at, updated_at, owner

Page:
- page_id
- title
- order
- fields[] (ordered)
- visibility (optional)

Field:
- field_id
- label
- type
- required (boolean)
- enum_ref (if enum types)
- validations (optional)
- visibility (optional)

Piece B — Form spec schema (JSON Schema)

File: `Axion/libraries/intake/schemas/intake_form_spec.v1.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "id": "intake_form_spec.v1",
 "type": "object",
 "required": ["form_id", "version", "pages", "created_at", "updated_at", "owner"],
 "properties": {
   "form_id": { "type": "string", "pattern": "^FORM-[A-Z0-9_]+$" },
   "version": { "type": "string", "minLength": 1 },

   "pages": {
     "type": "array",
     "minItems": 1,
     "items": { "$ref": "#/$defs/page" }
   },

   "created_at": { "type": "string", "format": "date-time" },
   "updated_at": { "type": "string", "format": "date-time" },
   "owner": { "type": "string", "minLength": 2 }
```

```
  },

  "$defs": {
    "page": {
      "type": "object",
      "required": ["page_id", "title", "order", "fields"],
      "properties": {
        "page_id": { "type": "string", "pattern": "^PAGE-[A-Z0-9_]+$" },
        "title": { "type": "string", "minLength": 2 },
        "order": { "type": "integer", "minimum": 1 },

        "visibility": { "$ref": "#/$defs/visibility_rule" },

        "fields": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/field" }
        }
      },
      "additionalProperties": false
    },

    "field": {
      "type": "object",
      "required": ["field_id", "label", "type", "required"],
      "properties": {
        "field_id": { "type": "string", "pattern": "^F-[A-Z0-9_]+$" },
        "label": { "type": "string", "minLength": 2 },

        "type": {
          "type": "string",
          "enum": ["text", "textarea", "number", "boolean", "enum", "multi_enum", "date", "file_ref",
"json"]
        },

        "required": { "type": "boolean" },

        "help_text": { "type": "string" },

        "enum_ref": { "type": "string", "description": "Required for enum/multi_enum" },

        "validations": {
          "type": "array",
          "items": { "$ref": "#/$defs/validation_rule" }
```

```
      },

      "visibility": { "$ref": "#/$defs/visibility_rule" }
    },
    "additionalProperties": false
  },

  "visibility_rule": {
    "type": "object",
    "required": ["mode"],
    "properties": {
      "mode": { "type": "string", "enum": ["always", "conditional"] },
      "when": {
        "type": "array",
        "description": "List of conditions (ANDed). Each condition references a field_id and
expected value.",
        "items": {
          "type": "object",
          "required": ["field_id", "op", "value"],
          "properties": {
            "field_id": { "type": "string" },
            "op": { "type": "string", "enum": ["==", "!=", "contains"] },
            "value": {}
          },
          "additionalProperties": false
        }
      }
    },
    "additionalProperties": false
  },

  "validation_rule": {
    "type": "object",
    "required": ["rule_id", "kind"],
    "properties": {
      "rule_id": { "type": "string", "pattern": "^VAL-[A-Z0-9_]+$" },
      "kind": {
        "type": "string",
        "enum": ["min_len", "max_len", "min", "max", "regex", "custom"]
      },
      "value": {},
      "message": { "type": "string" }
    },
    "additionalProperties": false
```

```
  }
 },

 "additionalProperties": false
}
```

Piece C — Determinism rules (ordering + versioning)

File: `Axion/libraries/intake/INT-1_determinism_rules.md`

```
---
library: intake
id: INT-1a
schema_version: 1.0.0
status: draft
---

# INT-1a — Determinism Rules (Form Spec)

- pages must be sorted by page.order (unique integers).
- fields order is the array order in the spec (stable).
- form spec changes require a version bump.
- visibility conditions are ANDed in listed order.
- field_id and page_id are immutable once published (deprecate via new version).
```

Piece D — Validation checklist

File: `Axion/libraries/intake/INT-1_validation_checklist.md`

```
---
library: intake
id: INT-1b
schema_version: 1.0.0
status: draft
---

# INT-1b — Validation Checklist

- [ ] form spec validates against intake_form_spec.v1 schema
- [ ] page.order values are unique and start at 1
- [ ] field_id values are unique across the spec
- [ ] enum_ref present for enum and multi_enum fields
```

- [ ] visibility rules reference existing field_id values

Field enums + allowed values registries

INT-2 — Field enums + allowed values registries

Piece A — Enum registry model (concept)

File: `Axion/libraries/intake/INT-2_enum_registry_model.md`

```
---
library: intake
id: INT-2
schema_version: 1.0.0
status: draft
---

# INT-2 — Enum Registry Model

## Purpose
Centralize allowed values for intake enum fields so:
- the wizard UI renders consistent options
- normalization can resolve aliases deterministically
- validation can reject unknown values consistently

## Core entities
- **Enum registry**: collection of enums by enum_id
- **Enum**: allowed values + labels + aliases

## Required behaviors
- enum values are canonical slugs (stable)
- aliases map to canonical values
- option order is stable and deterministic
```

Piece B — Enum registry schema (JSON Schema)

File: `Axion/libraries/intake/schemas/intake_enums.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "intake_enums.v1",
  "type": "object",
  "required": ["registry_id", "schema_version", "enums"],
  "properties": {
```

```
    "registry_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "enums": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/enum_def" }
    }
  },
  "$defs": {
    "enum_def": {
      "type": "object",
      "required": ["enum_id", "title", "options"],
      "properties": {
        "enum_id": { "type": "string", "pattern": "^ENUM-[A-Z0-9_]+$" },
        "title": { "type": "string", "minLength": 2 },
        "options": {
          "type": "array",
          "minItems": 1,
          "items": {
            "type": "object",
            "required": ["value", "label"],
            "properties": {
              "value": { "type": "string", "minLength": 1 },
              "label": { "type": "string", "minLength": 1 },
              "aliases": { "type": "array", "items": { "type": "string" } }
            },
            "additionalProperties": false
          }
        }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

Piece C — Starter enums registry (minimal set)

File: `Axion/libraries/intake/registries/intake_enums.v1.json`

```
{
  "schema_version": "1.0.0",
```

```
  "registry_id": "intake_enums.v1",
  "enums": [
    {
      "enum_id": "ENUM-RUN_PROFILE",
      "title": "Run Profile",
      "options": [
        { "value": "api", "label": "API", "aliases": ["backend", "server"] },
        { "value": "web", "label": "Web App", "aliases": ["frontend"] },
        { "value": "mobile", "label": "Mobile App", "aliases": ["ios", "android"] },
        { "value": "infra", "label": "Infrastructure", "aliases": ["devops"] },
        { "value": "data", "label": "Data", "aliases": ["analytics"] },
        { "value": "mixed", "label": "Mixed", "aliases": ["fullstack"] }
      ]
    },
    {
      "enum_id": "ENUM-RISK_CLASS",
      "title": "Risk Class",
      "options": [
        { "value": "prototype", "label": "Prototype", "aliases": ["poc", "mvp"] },
        { "value": "prod", "label": "Production", "aliases": ["production"] },
        { "value": "compliance", "label": "Compliance", "aliases": ["regulated"] }
      ]
    },
    {
      "enum_id": "ENUM-EXECUTOR_TYPE",
      "title": "Executor Type",
      "options": [
        { "value": "internal", "label": "Internal", "aliases": ["local"] },
        { "value": "external", "label": "External", "aliases": ["hosted"] }
      ]
    }
  ]
}
```

Piece D — Determinism rules (alias resolution)

File: `Axion/libraries/intake/INT-2_determinism_rules.md`

---
library: intake
id: INT-2a
schema_version: 1.0.0

status: draft

INT-2a — Determinism Rules (Enums)

- Enum option value is canonical.
- Aliases are case-insensitive and trimmed before matching.
- Resolution rule:
  1) exact match to canonical value
  2) else exact match to any alias
  3) else invalid value error
- Option order is the registry order and must not be re-sorted at runtime.

Piece E — Validation checklist

File: `Axion/libraries/intake/INT-2_validation_checklist.md`

---
library: intake
id: INT-2b
schema_version: 1.0.0
status: draft

INT-2b — Validation Checklist

- [ ] enum registry validates against intake_enums.v1 schema
- [ ] enum_id values are unique
- [ ] each enum option value is unique within its enum
- [ ] aliases do not collide across different canonical values in same enum
- [ ] alias resolution is deterministic (canonical → alias → error)

Say **“next”** for **INT-3 Validation rules (field-level + cross-field)**.

Validation rules

INT-3 — Validation rules (field-level + cross-field)

Piece A — Validation model (concept)

File: `Axion/libraries/intake/INT-3_validation_model.md`

```
---
library: intake
id: INT-3
schema_version: 1.0.0
status: draft
---

# INT-3 — Intake Validation Model

## Purpose
Define how intake submissions are validated consistently:
- field-level validation (type, required, regex, bounds)
- cross-field validation (dependencies and conditional requirements)
- stable error codes + pointers for UI and reports

## Validation surfaces
- **UI-time validation**: immediate feedback while filling the wizard
- **Submission validation (S2)**: authoritative validation for pipeline execution

## Output artifact
Validation produces an INTAKE_VALIDATION_REPORT artifact referenced by orchestration IO contracts.
```

Piece B — Cross-field rules schema

File: `Axion/libraries/intake/schemas/intake_cross_field_rules.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "intake_cross_field_rules.v1",
  "type": "object",
  "required": ["ruleset_id", "schema_version", "rules"],
  "properties": {
```

```
  "ruleset_id": { "type": "string", "minLength": 3 },
  "schema_version": { "type": "string", "minLength": 1 },
  "rules": {
    "type": "array",
    "minItems": 1,
    "items": { "$ref": "#/$defs/rule" }
  }
},
"$defs": {
  "rule": {
    "type": "object",
    "required": ["rule_id", "if", "then", "message"],
    "properties": {
      "rule_id": { "type": "string", "pattern": "^XVAL-[A-Z0-9_]+$" },

      "if": {
        "type": "array",
        "description": "AND conditions",
        "items": { "$ref": "#/$defs/condition" }
      },

      "then": {
        "type": "object",
        "required": ["require_fields"],
        "properties": {
          "require_fields": { "type": "array", "items": { "type": "string" } },
          "forbid_fields": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
      },

      "message": { "type": "string", "minLength": 3 }
    },
    "additionalProperties": false
  },

  "condition": {
    "type": "object",
    "required": ["field_id", "op", "value"],
    "properties": {
      "field_id": { "type": "string" },
      "op": { "type": "string", "enum": ["==", "!=", "contains", "exists"] },
      "value": {}
    },
```

```
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

Piece C — Validation report schema

File:
`Axion/libraries/intake/schemas/intake_validation_report.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "intake_validation_report.v1",
  "type": "object",
  "required": ["submission_id", "status", "errors", "warnings", "validated_at"],
  "properties": {
    "submission_id": { "type": "string" },
    "status": { "type": "string", "enum": ["pass", "fail", "warn"] },

    "errors": {
      "type": "array",
      "items": { "$ref": "#/$defs/issue" }
    },
    "warnings": {
      "type": "array",
      "items": { "$ref": "#/$defs/issue" }
    },

    "validated_at": { "type": "string", "format": "date-time" }
  },
  "$defs": {
    "issue": {
      "type": "object",
      "required": ["code", "field_id", "message"],
      "properties": {
        "code": { "type": "string", "pattern": "^INT-[A-Z0-9_]+$" },
        "field_id": { "type": "string" },
        "message": { "type": "string" },
        "pointer": { "type": "string", "description": "Optional JSON pointer into submission record" }
      },
```

```
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

Piece D — Starter cross-field rules (example set)

File: `Axion/libraries/intake/registries/intake_cross_field_rules.v1.json`

```
{
  "schema_version": "1.0.0",
  "ruleset_id": "intake_cross_field_rules.v1",
  "rules": [
    {
      "rule_id": "XVAL-REQUIRE_STACK_FOR_WEB",
      "if": [
        { "field_id": "F-RUN_PROFILE", "op": "==", "value": "web" }
      ],
      "then": { "require_fields": ["F-FRONTEND_STACK"] },
      "message": "Frontend stack is required for web runs."
    },
    {
      "rule_id": "XVAL-EXTERNAL_EXECUTOR_REQUIRES_REPO",
      "if": [
        { "field_id": "F-EXECUTOR_TYPE", "op": "==", "value": "external" }
      ],
      "then": { "require_fields": ["F-REPO_URL"] },
      "message": "Repo URL is required for external execution."
    }
  ]
}
```

Piece E — Determinism rules (errors + ordering)

File: `Axion/libraries/intake/INT-3_determinism_rules.md`

```
---
library: intake
id: INT-3a
```

schema_version: 1.0.0
status: draft

INT-3a — Determinism Rules (Validation)

- Validation runs in fixed order:
  1) schema/type checks
  2) required fields
  3) enum resolution
  4) field-level validators (regex/bounds)
  5) cross-field rules
- Error codes are stable and prefixed `INT-...`.
- Output ordering:
  - errors sorted by field_id, then code
  - warnings sorted by field_id, then code
- Pointer format is JSON pointer into submission record.

Piece F — Validation checklist

File: `Axion/libraries/intake/INT-3_validation_checklist.md`

---
library: intake
id: INT-3b
schema_version: 1.0.0
status: draft

INT-3b — Validation Checklist

- [ ] cross-field rules validate against schema
- [ ] validation report validates against schema
- [ ] error codes use INT- prefix and are stable
- [ ] deterministic ordering applied (errors/warnings sorted)
- [ ] cross-field rules reference existing field IDs from the form spec (strict mode)

Submission record schema

INT-4 — Submission record schema (raw + normalized)

Piece A — Submission record model (concept)

File: `Axion/libraries/intake/INT-4_submission_record_model.md`

```
---
library: intake
id: INT-4
schema_version: 1.0.0
status: draft
---

# INT-4 — Submission Record Model

## Purpose
Define the canonical record for intake submissions:
- the raw answers as entered (auditable)
- normalized answers (canonicalized, enum-resolved, stable ordering)
- metadata required for deterministic downstream processing

## Artifacts
- INTAKE_SUBMISSION: raw submission record (wizard output)
- NORMALIZED_INPUT: normalized artifact used downstream (S1 output)

## Principles
- Keep raw and normalized separate.
- Never overwrite raw answers.
- Normalized output is deterministic from raw + pinned form spec + pinned enum registry +
pinned normalization rules.
```

Piece B — Intake submission schema (raw)

File: `Axion/libraries/intake/schemas/intake_submission.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "intake_submission.v1",
  "type": "object",
  "required": ["submission_id", "form_ref", "submitted_at", "answers"],
```

```
  "properties": {
    "submission_id": { "type": "string", "pattern": "^SUB-[A-Z0-9]{6,}$" },

    "form_ref": {
      "type": "object",
      "required": ["form_id", "version"],
      "properties": {
        "form_id": { "type": "string" },
        "version": { "type": "string" }
      },
      "additionalProperties": false
    },

    "submitted_at": { "type": "string", "format": "date-time" },

    "actor": {
      "type": "object",
      "properties": {
        "actor_id": { "type": "string" },
        "role": { "type": "string" }
      },
      "additionalProperties": false
    },

    "answers": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["field_id", "value"],
        "properties": {
          "field_id": { "type": "string" },
          "value": {},
          "raw_text": { "type": "string", "description": "Optional original user text for
enums/normalization auditing" }
        },
        "additionalProperties": false
      }
    },

    "attachments": {
      "type": "array",
      "items": {
        "type": "object",
```

```
      "required": ["attachment_id", "kind", "path"],
      "properties": {
        "attachment_id": { "type": "string", "pattern": "^ATT-[A-Z0-9]{6,}$" },
        "kind": { "type": "string", "enum": ["file", "zip", "url_ref"] },
        "path": { "type": "string" },
        "hash": { "type": "string" }
      },
      "additionalProperties": false
    }
  }
},
"additionalProperties": false
}
```

Piece C — Normalized input schema (canonical artifact)

File: `Axion/libraries/intake/schemas/normalized_input.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "normalized_input.v1",
  "type": "object",
  "required": [
    "submission_id",
    "normalized_at",
    "inputs",
    "derived",
    "provenance"
  ],
  "properties": {
    "submission_id": { "type": "string" },
    "normalized_at": { "type": "string", "format": "date-time" },

    "inputs": {
      "type": "object",
      "description": "Canonicalized input values used downstream",
      "additionalProperties": true
    },

    "derived": {
      "type": "object",
```

```
      "description": "Derived/normalized helpers (slugs, parsed lists, flags)",
      "additionalProperties": true
    },

    "provenance": {
      "type": "object",
      "required": ["form_ref", "enum_registry_ref", "normalization_rules_ref"],
      "properties": {
        "form_ref": {
          "type": "object",
          "required": ["form_id", "version"],
          "properties": { "form_id": { "type": "string" }, "version": { "type": "string" } },
          "additionalProperties": false
        },
        "enum_registry_ref": {
          "type": "object",
          "required": ["registry_id", "schema_version"],
          "properties": { "registry_id": { "type": "string" }, "schema_version": { "type": "string" } },
          "additionalProperties": false
        },
        "normalization_rules_ref": {
          "type": "object",
          "required": ["rules_id", "schema_version"],
          "properties": { "rules_id": { "type": "string" }, "schema_version": { "type": "string" } },
          "additionalProperties": false
        }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

Piece D — Determinism rules (raw → normalized)

File: `Axion/libraries/intake/INT-4_determinism_rules.md`

```
---
library: intake
id: INT-4a
schema_version: 1.0.0
status: draft
```

INT-4a — Determinism Rules (Submission → Normalized)

Normalization must be a pure function of:
- intake_submission (raw)
- pinned form spec (form_id + version)
- pinned enum registry (registry_id + schema_version)
- pinned normalization rules

Rules:
- answers array is normalized into inputs map keyed by canonical field_id
- enum values are alias-resolved to canonical (INT-2)
- multi-enum values are de-duplicated and sorted lexicographically
- whitespace trimmed; internal whitespace collapsed (where applicable)
- derived values (like slugs) use a stable slugify rule
- provenance refs recorded in normalized_input

Piece E — Validation checklist

File: `Axion/libraries/intake/INT-4_validation_checklist.md`

---
library: intake
id: INT-4b
schema_version: 1.0.0
status: draft

INT-4b — Validation Checklist

- [ ] intake_submission validates against schema
- [ ] normalized_input validates against schema
- [ ] normalized_input.provenance is present and matches pinned refs
- [ ] enum resolution applied consistently (canonical/alias)
- [ ] multi-select outputs are sorted and de-duped
- [ ] raw answers are preserved (no overwrites)

.

Determinism rules

INT-5 — Determinism rules (normalization, stable IDs, ordering)

Piece A — Normalization rules registry (machine-readable)

File: `Axion/libraries/intake/schemas/normalization_rules.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "normalization_rules.v1",
  "type": "object",
  "required": ["rules_id", "schema_version", "rules"],
  "properties": {
    "rules_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "rules": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["rule_id", "applies_to", "transform"],
        "properties": {
          "rule_id": { "type": "string", "pattern": "^NORM-[A-Z0-9_]+$" },
          "applies_to": {
            "type": "object",
            "required": ["field_ids"],
            "properties": {
              "field_ids": { "type": "array", "items": { "type": "string" }, "minItems": 1 }
            },
            "additionalProperties": false
          },
          "transform": {
            "type": "object",
            "required": ["kind"],
            "properties": {
              "kind": {
                "type": "string",
                "enum": [
                  "trim",
                  "collapse_whitespace",
                  "lowercase",
```

```
                "uppercase",
                "slugify",
                "dedupe_sort",
                "parse_csv_list"
              ]
            }
          },
          "additionalProperties": false
        }
      },
      "additionalProperties": false
    }
  }
},
"additionalProperties": false
}
```

Piece B — Starter normalization rules (registry)

File: `Axion/libraries/intake/registries/normalization_rules.v1.json`

```
{
  "schema_version": "1.0.0",
  "rules_id": "normalization_rules.v1",
  "rules": [
    {
      "rule_id": "NORM-TRIM-ALL",
      "applies_to": { "field_ids": ["*"] },
      "transform": { "kind": "trim" }
    },
    {
      "rule_id": "NORM-COLLAPSE-WS",
      "applies_to": { "field_ids": ["*"] },
      "transform": { "kind": "collapse_whitespace" }
    },
    {
      "rule_id": "NORM-SLUGIFY-PROJECT",
      "applies_to": { "field_ids": ["F-PROJECT_NAME"] },
      "transform": { "kind": "slugify" }
    },
    {
      "rule_id": "NORM-DEDUPE-SORT-MULTI",
```

```
    "applies_to": { "field_ids": ["*MULTI_ENUM*"] },
    "transform": { "kind": "dedupe_sort" }
  }
 ]
}
```

Notes:

- `"*"` and `"*MULTI_ENUM*"` are placeholders; the engine can interpret these as match patterns OR you can require explicit field lists only. If you want strict-only, remove wildcards and list field_ids explicitly.

Piece C — Stable ID rules (submission_id, attachment_id)

File: `Axion/libraries/intake/INT-5_stable_id_rules.md`

```
---
library: intake
id: INT-5
schema_version: 1.0.0
status: draft
---

# INT-5 — Stable ID + Ordering Rules

## Stable IDs
### submission_id
- Format: SUB-<6+ alnum>
- Must be generated once at submission creation.
- Never regenerated during normalization or validation.

### attachment_id
- Format: ATT-<6+ alnum>
- Stable for the life of the submission.
- Attachment hash is optional but recommended.

## Ordering rules
- intake_submission.answers must be sorted by field_id (for deterministic storage)
  - OR preserve user order but normalized_input must be deterministic. Pick one; recommended:
    - keep raw answers in user order (audit)
    - normalized_input uses deterministic map + sorted multi-values
- normalized_input multi lists are:
```

- trimmed
- de-duped
- lexicographically sorted

Slugify rule (stable)
- lowercase
- spaces/underscores → hyphen
- remove non-alnum except hyphen
- collapse repeated hyphens

Piece D — Determinism checklist

File: `Axion/libraries/intake/INT-5_determinism_checklist.md`

---
library: intake
id: INT-5a
schema_version: 1.0.0
status: draft

INT-5a — Determinism Checklist

- [ ] Normalization rules are pinned via provenance refs
- [ ] Enum alias resolution is applied before any downstream use
- [ ] Multi-value fields are de-duped + sorted
- [ ] Slugify is stable and documented
- [ ] IDs are generated once and never regenerated

Piece E — Validation checklist

File: `Axion/libraries/intake/INT-5_validation_checklist.md`

---
library: intake
id: INT-5b
schema_version: 1.0.0
status: draft

INT-5b — Validation Checklist

- [ ] normalization_rules registry validates against schema
- [ ] normalization rules referenced in normalized_input.provenance
- [ ] deterministic outputs verified (same input → same normalized)
- [ ] stable ID rules followed for submission and attachments

Gates

INT-6 — Gates (validity, required fields, normalization pass)

Piece A — Intake gate intent

File: `Axion/libraries/intake/INT-6_intake_gates.md`

```
---
library: intake
id: INT-6
schema_version: 1.0.0
status: draft
---

# INT-6 — Intake Gates

## Purpose
Enforce that downstream stages only run on valid, deterministic inputs.

## Gate set (minimum)

### INT-GATE-01 — Submission schema valid
- intake_submission validates against intake_submission.v1 schema

### INT-GATE-02 — Form spec pinned + compatible
- submission.form_ref exists and matches a pinned intake_form_spec version

### INT-GATE-03 — Enum resolution valid
- enum and multi_enum answers resolve to canonical values (aliases supported)
- unknown enum values → errors

### INT-GATE-04 — Required fields satisfied
- all required fields present (including conditional requirements)

### INT-GATE-05 — Cross-field rules satisfied
- cross-field rules evaluated deterministically
- missing required fields flagged with stable error codes

### INT-GATE-06 — Normalization output valid
- normalized_input produced
- normalized_input validates against normalized_input.v1 schema
- normalized_input.provenance refs are present and pinned
```

Output
- intake_validation_report artifact must be produced for S2
- status:
  - fail if any INT-GATE-* hard rule fails
  - warn if non-blocking warnings only (optional)

Piece B — Mapping to pipeline gate (G1_INTAKE_VALIDITY)

File: `Axion/libraries/intake/INT-6_gate_mapping.md`

---
library: intake
id: INT-6a
schema_version: 1.0.0
status: draft

INT-6a — Mapping to Pipeline Gates

Pipeline gate: **G1_INTAKE_VALIDITY**

G1 passes if:
- INT-GATE-01..06 all pass

G1 fails/pause if:
- any INT-GATE hard rule fails
- output must include intake_validation_report with error list + pointers

---

Piece C — Machine-readable gate spec (optional; if you encode as rules)

File: `Axion/libraries/intake/INT-6_intake_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "INT-6b",
  "gates": [
    {
      "gate_id": "INT-GATE-01",
      "name": "Submission schema valid",
```

```
    "severity": "hard_stop",
    "checks": ["schema_valid(intake_submission.v1)"]
  },
  {
    "gate_id": "INT-GATE-02",
    "name": "Form spec pinned + compatible",
    "severity": "hard_stop",
    "checks": ["submission.form_ref matches pinned_form_spec"]
  },
  {
    "gate_id": "INT-GATE-03",
    "name": "Enum resolution valid",
    "severity": "hard_stop",
    "checks": ["all enum answers resolve via intake_enums registry"]
  },
  {
    "gate_id": "INT-GATE-04",
    "name": "Required fields satisfied",
    "severity": "hard_stop",
    "checks": ["required fields present including conditional requirements"]
  },
  {
    "gate_id": "INT-GATE-05",
    "name": "Cross-field rules satisfied",
    "severity": "hard_stop",
    "checks": ["cross-field rules evaluated; no violations"]
  },
  {
    "gate_id": "INT-GATE-06",
    "name": "Normalization output valid",
    "severity": "hard_stop",
    "checks": ["normalized_input exists and schema_valid(normalized_input.v1)",
"normalized_input.provenance present"]
  }
 ]
}
```

Piece D — Evidence requirements (what failures must include)

File: `Axion/libraries/intake/INT-6_evidence_requirements.md`

---

library: intake
id: INT-6c
schema_version: 1.0.0
status: draft

INT-6c — Evidence Requirements

For any failure, intake_validation_report must include:
- code (INT-...)
- field_id
- message
- pointer (JSON pointer into submission)

For cross-field failures:
- include rule_id (XVAL-...) in message or details
- pointers should include the triggering field(s) and missing/invalid field(s)

Piece E — Validation checklist

File: `Axion/libraries/intake/INT-6_validation_checklist.md`

---
library: intake
id: INT-6d
schema_version: 1.0.0
status: draft

INT-6d — Validation Checklist

- [ ] INT gates cover schema, pins, enums, required fields, cross-field, normalization
- [ ] intake_validation_report emitted on failure with stable codes + pointers
- [ ] G1 mapping is explicit (INT gates → G1)
- [ ] normalized_input provenance is required and pinned

Say **“next”** for **INT-7 Minimum viable intake set**.

Minimum viable intake

INT-7 — Minimum viable intake set (runtime-required files + “done”)

Piece A — MV intake inventory

File: `Axion/libraries/intake/INT-7_minimum_viable_set.md`

---
library: intake
id: INT-7
schema_version: 1.0.0
status: draft

INT-7 — Minimum Viable intake/ Set

Goal
Define the smallest intake library set required to collect inputs and produce deterministic normalized artifacts.

Required runtime files (minimum)
1) Core docs
- INT-0_purpose.md
- INT-0_boundary_checklist.md
- INT-1_form_spec_model.md
- INT-1_determinism_rules.md
- INT-1_validation_checklist.md
- INT-2_enum_registry_model.md
- INT-2_determinism_rules.md
- INT-2_validation_checklist.md
- INT-3_validation_model.md
- INT-3_determinism_rules.md
- INT-3_validation_checklist.md
- INT-4_submission_record_model.md
- INT-4_determinism_rules.md
- INT-4_validation_checklist.md
- INT-5_stable_id_rules.md
- INT-5_determinism_checklist.md
- INT-5_validation_checklist.md
- INT-6_intake_gates.md
- INT-6_gate_mapping.md
- INT-6_evidence_requirements.md

- INT-6_validation_checklist.md
- INT-7_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/intake_form_spec.v1.schema.json
- schemas/intake_enums.v1.schema.json
- schemas/intake_cross_field_rules.v1.schema.json
- schemas/intake_submission.v1.schema.json
- schemas/normalized_input.v1.schema.json
- schemas/normalization_rules.v1.schema.json
- schemas/intake_validation_report.v1.schema.json

3) Registries (runtime defaults)
- registries/intake_enums.v1.json
- registries/intake_cross_field_rules.v1.json
- registries/normalization_rules.v1.json
- registries/form_spec.axion.v1.json (your actual wizard spec; created from INT-1)
4) Gate specs (optional)
- INT-6_intake_gates.spec.json

Piece B — Definition of done (intake/)

File: `Axion/libraries/intake/INT-7_definition_of_done.md`

---
library: intake
id: INT-7a
schema_version: 1.0.0
status: draft

INT-7a — intake/ Definition of Done

intake/ is “done” when:

Schemas + registries
- [ ] All intake schemas validate (JSON Schema check)
- [ ] intake_enums registry validates and covers required enums
- [ ] cross-field rules registry validates
- [ ] normalization rules registry validates
- [ ] form spec exists, validates, and is versioned

Runtime behavior (contract-level)
- [ ] Wizard can render the form spec deterministically (pages/fields order stable)
- [ ] Submission record (raw) is created and validates
- [ ] Validation produces an intake_validation_report with stable codes + pointers
- [ ] Normalization produces normalized_input deterministically with provenance refs

Gates
- [ ] INT-GATE-01..06 enforce schema, pins, enums, required fields, cross-field rules, normalization
- [ ] Pipeline gate G1_INTAKE_VALIDITY is satisfied when all INT gates pass

---

Piece C — Minimal folder tree

File: `Axion/libraries/intake/INT-7_minimal_tree.txt`

```
Axion/libraries/intake/
  INT-0_purpose.md
  INT-0_boundary_checklist.md
  INT-1_form_spec_model.md
  INT-1_determinism_rules.md
  INT-1_validation_checklist.md
  INT-2_enum_registry_model.md
  INT-2_determinism_rules.md
  INT-2_validation_checklist.md
  INT-3_validation_model.md
  INT-3_determinism_rules.md
  INT-3_validation_checklist.md
  INT-4_submission_record_model.md
  INT-4_determinism_rules.md
  INT-4_validation_checklist.md
  INT-5_stable_id_rules.md
  INT-5_determinism_checklist.md
  INT-5_validation_checklist.md
  INT-6_intake_gates.md
  INT-6_gate_mapping.md
  INT-6_evidence_requirements.md
  INT-6_validation_checklist.md
  INT-7_minimum_viable_set.md
  INT-7_definition_of_done.md
  INT-7_minimal_tree.txt

  schemas/
```

```
  intake_form_spec.v1.schema.json
  intake_enums.v1.schema.json
  intake_cross_field_rules.v1.schema.json
  intake_submission.v1.schema.json
  normalized_input.v1.schema.json
  normalization_rules.v1.schema.json
  intake_validation_report.v1.schema.json

registries/
  form_spec.axion.v1.json
  intake_enums.v1.json
  intake_cross_field_rules.v1.json
  normalization_rules.v1.json
```